Arrow detection in diagrams using VLLMs

Luca Giacinto Cicchese
Projectwork in NLP, Universita' degli Studi di Firenze

Abstract

This projectwork is about detecting arrows in diagrams using a vision large language model (VLLM) instead of a classic object detector. It's an interesting task to hand an LLM as spatial awareness is exactly the kind of thing these models aren't built for. Before committing to this approach I ran a test on Claude, Gemini and GPT to understand if the task can be solved by the most advanced models available. All three handled the task reasonably well, but those are huge closed models. The model used for the rest of the project is Qwen3-VL-Instruct at 8B parameters. Its zero-shot performance is exactly what we would expect: it may detect how many arrows are there in the image but it can't detect their position at all. Getting it anywhere close to usable took many NLP techniques: reformulating the output as normalized point coordinates instead of raw pixels, LoRA fine-tuning, a Number Token Loss on top of the usual cross-entropy, reordering the detected arrows, using boxes to guide the output and self-consistency at inference time. Together these push the model to perform more than seven times better than the starting point. It's still behind a specialized detector like YOLO, but the improvement over its own zero-shot performance is significant. The code for the project is available at `https://github.com/lucacicchese/NLP_projectwork`

1 Introduction

Detecting arrows in diagrams is a classic object detection problem. My goal was to see if giving a VLLM a picture of a diagram and a prompt it could give out the coordinates of every arrow in it without the use of other external tools.

The usual way to solve this kind of task is to use a specialized detector like YOLO finetuned on a dataset containing the object we are trying to detect. For this reason this is the first baseline test I ran. I found a YOLO checkpoint that had been trained to detect elements of diagrams including arrows [7] and ran it on the validation set that I used for the rest of the project. The authors of this model declare a precision of up to 95% on arrows [8], measured on their own test set and with a different metric from the one I used, so these numbers can't be compared directly with my results. As expected YOLO performed extremely well, even though it sometimes missed some arrows or detected other objects as arrows. The main problem when using specialized detectors like this is that they require to be retrained to detect new kinds of objects. On the other hand a VLLM, once it understands the task and the space can be repurposed to detect something completely different without retraining.

There's a real difference in how the two approaches fail. YOLO regresses box coordinates directly, so there's no real notion of "close but wrong" for it: a box is either there or it isn't. The VLLM, as we'll see later (Figure 4), tends to fail the other way around: it usually finds something, just not always in exactly the right spot.

Choosing a VLLM or a specialized detector has obvious trade-offs. A specialized detector is more accurate and faster but a VLLM is more flexible. The real question is, how much this flexibility costs in terms of accuracy. We also know from the initial tests that having a big model can overcome most of the accuracy gap but my goal is to see how close I can get with a relatively small model.

Before starting the actual project I ran an informal test of this task on the current Claude, Gemini and GPT models through arena.ai, mostly out of curiosity, and the arrows they picked out were noticeably better than anything I got later out of Qwen. One thing stood out in particular: the "thinking" variants of these models consistently did better than their non-thinking counterparts, which makes some sense if part of this task genuinely benefits from working through the geometry step by step rather than pattern-matching it in a single pass. None of this is a fair comparison, of course (those are closed, frontier-scale models against an 8B open one), but it set my expectations going in: Qwen3-VL-Instruct was going to need real help to be usable at all. That's really the question this project tries to answer: how far can a small, self-hostable model be pushed with the right combination of techniques, not whether it can beat something ten or a hundred times its size.

2 Datasets

There availability of datasets to train this task is extremely low, especially labelled datasets with correct labels. For this reason after analyzing a few alternatives I decided to use a combination of a real dataset (FC-Detection) and a synthetic one FlowLearn. This combination of datasets allowed me to train on as much data as I needed but still gaining real world performance.

2.1 FC-Detection

The real dataset used for training and validation is FC-Detection [5]. It has 69 training images and 30 validation images. Every image of the dataset has bounding boxes for each element of the flowchart including arrows. For

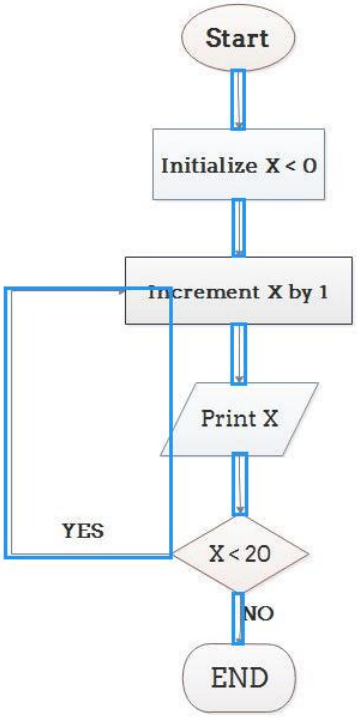


Figure 1: YOLO on a linear flowchart. The straight vertical arrows are detected precisely, but the feedback arrow on the left, which goes back up to "Increment X by 1", is not detected at all.

each arrow there are both a regular bounding box and two additional coordinates for the start and end of the arrow. The start and end coordinates resulted better to use than bounding boxes in this case since only four coordinates are needed to describe an arrow and the model can learn to predict them more accurately. I came to this conclusion after experimenting with the full bounding boxes and noticing that the model tended to give coordinates that weren't aligned as expected or that didn't even form a rectangle. By using the start and end coordinates we also avoid degenerate bounding boxes.

2.2 FlowLearn

To increase the small amount of real data available, I used FlowLearn [4] to generate synthetic flowcharts. FlowLearn allows to generate flowcharts in two different styles, word and char, so I used both for added variety. For each type of flowchart I generated 600 images, for a total of 1200 synthetic flowcharts. These images were used for the first stage of training where the goal was to teach the model to know exactly what an arrow is and how to find locate it in the image.

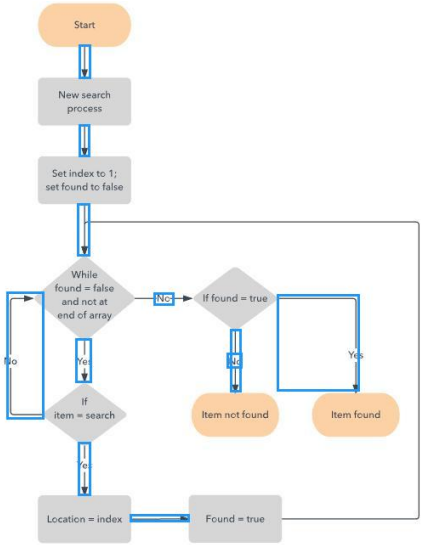


Figure 2: YOLO on a more complex flowchart with nested loops. The straight arrows are still detected well, the two smaller loops are only partially detected with broken boxes, and the long outer loop from "Found = true" to the top is completely missed.

3 Methodology

After an initial zero-shot test on the validation set I started testing a few different techniques to improve the model's performance. I started with prompt engineering, then added LoRA fine tuning. Since the results were still not satisfactory I thought about what could be the reason and my guess was that the loss was considering the numbers as tokens and not as numbers. Looking at the literature I found the Number Token Loss (NTL) which is a loss function that takes into account the numerical value of the tokens. Since the model could still use an improvement I added a self-consistency technique to get better predictions.

3.1 Task identification and prompt engineering

The first step I focused on was making sure the task was well defined, that there was no ambiguity in the prompt and that the model understood what it should do. I went through multiple iterations of prompt engineering both manually and with the help of an LLM until I got a prompt that was capable of getting consistent results. The final prompt is shown in the box below but the idea was to make the model find the arrowheads first and then trace back until the end of the tail, this way curved arrows would have been easier to detect. I also clearly explained how the JSON output should be structured, using the conventions already present in the Qwen training. I also found out to be necessary to normalize the coordinates as the model got confused by images of different sizes and tended to put all arrows in a small area of the image.

```
"Look for arrowheads in this image.  For each
one, find its point location as a 2d point, then
from that arrowhead find its tail.  Do this for
every arrow.

Express every coordinate on a normalized 0-1000
scale, independent of the image's actual pixel
resolution:  x = round(pixel_x / image_width *
1000), y = round(pixel_y / image_height * 1000).
So (0, 0) is the top-left corner and (1000,
1000) is the bottom-right corner of the image,
regardless of its real size.

Only output the coordinates in this normalized
format, linking each tail and head pair with a
label:  [{"point_2d":  [x, y], "label":
"arrow_1_tail"}, {"point_2d":  [x, y], "label":
"arrow_1_head"}, ...]

If there are no arrows, respond with []."
```

3.2 Parameter Efficient Fine Tuning (LoRA)

Of course prompt engineering alone isn't enough to make a small model learn a completely new task. It is also impossible, and counterproductive, to fine tune the whole model. The solution is to use a parameter efficient fine tuning technique, I used LoRA [1] as it is widely used and was already familiar with it. This technique allows to train only a small number of parameters that get multiplied to the original weights of the model. This allow to have a small checkpoint that can be loaded on top of the original model. I adjusted the LoRA rank to find the best trade off and ended up using r=48, after starting at r=16 and raising it once I understood that the model wasn't able to learn the task for lack of capacity.

3.3 Data augmentation

Since all labelled datasets found online for this task are small I decided to train the LoRA adapter in two stages. The first stage was trained on synthetic data from FlowLearn since I could generate as much data as I wanted. The second stage was trained on the real data from FC_Detection to improve the model's accuracy on real images.

3.4 Number Token Loss (NTL)

LLMs in general are very good at predicting tokens but for this kind of models a number is just a token like all the others. That means that the model doesn't understand that the distance between two numbers is meaningful, and in a task like this where I'm trying to get exact coordinates that's extremely important. Not acknowledging this issue means that when I'm finetuning the model and the output is wrong the model only gets a signal that it chose the wrong token but not if it was close or far from the target. For this reason I consulted the literature and found the Number Token Loss [2] which is a loss function that takes into account the numerical value of the tokens. I then weighted the NTL loss with the cross entropy loss to get a better signal for the model when it makes a mistake.

$$\mathcal{L} = \mathcal{L}_{CE} + 0.3 \cdot \mathcal{L}_{NTL}$$

3.5 Self-Consistency

After many iterations of fine-tuning and loss shaping, the model was still making a lot of mistakes. I decided at this point to try another technique that could improve the results without retraining the model. I used the self-consistency technique [3] to improve the results at inference time. The idea is to vary the temperature of the model and sample the output multiple times then take the majority vote of the results to make the final prediction more robust.

4 Results

Since the model outputs the tail and head points of each arrow, I chose to evaluate it by comparing these points directly with the ones in the ground truth. I used a point-to-point metric similar to PCK (Percentage of Correct Keypoints): a predicted point is counted as correct if it is within 3% of the image diagonal from the matching arrow_start or arrow_end point of the ground truth. This way the evaluation checks exactly what the model has to do, which is to generate the right coordinates for each arrow.

In practice the evaluation works like this. The tails predicted by the model are compared only with the arrow_start points of the ground truth, and the heads only with the arrow_end points, so an arrow predicted in the wrong direction is counted as an error. For every predicted point I look for the closest ground truth point that is still free: if it is within the tolerance it counts as a true positive and that ground truth point can't be used again, otherwise the prediction is a false positive. The ground truth points left without a match at the end are the false negatives. The tolerance depends on the size of the image. Each arrow counts twice, once for the tail and once for the head, so an arrow with the right head and the wrong tail is half correct. The counts are summed over all the images and only then I computed the three metrics: precision tells how many of the predicted points are correct, recall tells how many of the real points the model found, and F1 combines the two in a single number.

It is important to note that these metrics are used only to evaluate the model and are not part of the training. The loss described in the Methodology section works on the tokens of the JSON output: the cross entropy checks if every token, numbers included, is exactly the one in the target, while the NTL adds a signal on how far each digit is from the right one. This is only an approximation of

the distance in pixels, since the NTL looks at one digit at a time: if the target is 345, predicting 445 or 346 gives the same NTL even if the first is 100 units away and the second only 1. The loss also expects the arrows in the same order as the label file, while the metric doesn't care about the order. This means that a lower loss usually brings better metrics, but the two don't always move together.

Configuration	Prec.	Recall	F1
Zero-shot (no fine-tuning)	0.170	0.068	0.097
Baseline (absolute coords.)	0.289	0.291	0.290
+ scale/direction fix	0.379	0.353	0.365
+ FlowLearn data (r=16)	0.356	0.445	0.395
+ LoRA rank 32	0.449	0.434	0.441
+ LoRA rank 48, more data	0.516	0.414	0.459
+ Self-Consistency (N=5)	**0.553**	**0.516**	**0.534**
− NTL (rank 48, greedy)	0.369	0.151	0.214

Table 1: Evolution of the metrics on the validation set (30 images) across the project iterations. Precision/Recall/F1 computed with point-to-point matching (tolerance: 3% of the image diagonal).

The zero-shot test is the starting point of the project. Without any fine-tuning Qwen3-VL-Instruct gets only about one detection in six right, and the recall is even worse at 0.068. Most arrows are not found at all, with 48 true positives against 660 false negatives. This is in line with the informal tests I did on Claude, Gemini and GPT: a small 8B model without any adaptation to the task can't compete with the big closed models on spatial tasks.

All the configurations from "Baseline (absolute coords.)" onward use LoRA fine-tuning, and the NTL was part of the training loss from this first fine-tuned configuration onward. Getting there brought F1 from 0.097 to 0.290, roughly 3 times better, and this is the biggest single gain in the whole table. After that the improvements get smaller. Fixing the scale and direction of the coordinates improved F1 by another 26% (from 0.290 to 0.365). This showed me that a good part of the early errors came from a bug in how the coordinates were handled and not from the model being unable to do the task. Adding the FlowLearn data at rank 16 increased the recall by 26% but lowered the precision a bit: the model found more arrows but also produced more false positives, probably because before it hadn't seen enough variety of layouts. Raising the LoRA rank from 16 to 32 on the same data recovered the precision (+26%) without losing much of the recall, which confirms that at rank 16 the model was limited by its capacity and not by the data. Going to rank 48 with more data and more epochs still helped, but less than before: +4% F1 against the +12% of the previous step, so this combination of rank and data was close to its limit.

Self-consistency improved both precision and recall at the same time (+16% F1 over the previous checkpoint), after the last two steps had increased the precision while lowering the recall. It is also the only technique that doesn't change the model at all, since it works only at inference time and doesn't need any retraining. Overall F1 went from 0.097 with the zero-shot model to 0.534 with the final pipeline, more than five times better. None of the single steps is a big improvement on its own, but together they turn a model that was unusable into one that works reasonably well, without making the model any bigger.

4.1 Ablation: Number Token Loss

To measure the actual contribution of the NTL, I trained the final configuration again (rank 48, same data, same two stages with 5 epochs on FlowLearn and 3 on FC-Detection) using only the cross entropy loss. I then evaluated this model and the full one in the same run. In this run the full configuration gets an F1 of 0.439 with greedy decoding and 0.508 with self-consistency.

Removing the NTL halves the F1: it goes from 0.439 to 0.214 with greedy decoding (last row of Table 1) and from 0.508 to 0.237 with self-consistency. Most of the drop is in the recall, from 0.414 to 0.151. Precision also gets worse (from 0.468 to 0.369).

I expected NTL to have had a big impact on the metrics and this ablation confirms that it has the biggest effect of all techniques used in this project.

4.2 Impact and variance of self-consistency

To analyze the impact of self-consistency I analyzed the variance between different runs on the best checkpoint reached during training. I ran the model on the validation set with a temperature of 0.7 and sampled 5 outputs per image, then computed the consensus of the 5 outputs with a voting threshold of 2, meaning that an arrow is kept if it appears in at least 2 of the 5 samples. This configuration gave an F1 of 0.541, +0.05 over greedy decoding on the same checkpoint.

The first result is that a single sampled output is noisy. Treating each of the 10 samples as an independent run gives an F1 of 0.463 ± 0.023 (from 0.425 to 0.488), while the greedy output of the same run reaches 0.492, so sampling alone makes the predictions slightly worse and the gain comes only from aggregating them. The samples also disagree a lot with each other: two samples of the same image share on average only 42% of their arrows, the number of predicted arrows changes by about two arrows per image, and in no image did all 10 samples agree on how many arrows there are. Of all the arrows found across the samples, 61% appear in a single sample and only 2.6% appear in all ten.

When the samples do agree on an arrow, though, they also agree on where it is: the points grouped together are on average less than 1% of the image diagonal away from

the group center, well below the 3% tolerance. So the variance is mostly about which arrows the model writes down and not about where it puts them.

Inference	Prec.	Recall	F1
Greedy	0.553	0.444	0.492
Single sample (mean of 10)	0.480	0.448	0.463
N=5, ≥2 of 5	0.539	0.542	**0.541**
N=5, ≥3 of 5	0.685	0.340	0.455
N=7, union	0.237	0.768	0.362
N=10, all 10	0.936	0.103	0.186

Table 2: Self-consistency on the validation set with different numbers of samples and voting thresholds (an arrow is kept if it appears in at least the given number of samples).

The voting threshold controls a clear trade-off between precision and recall (Table 2). With N=5 and an arrow kept when it appears in at least 2 of the 5 samples, F1 is 0.541, +0.05 over greedy. Requiring an arrow to appear in all 10 samples brings the precision to 0.936 but the recall drops to 0.103, so the agreement between samples works well as a confidence score for each arrow. Going beyond 5 samples doesn't help, as F1 stays between 0.50 and 0.54. In this run the gain over greedy comes entirely from the recall, while the precision stays at the same level.

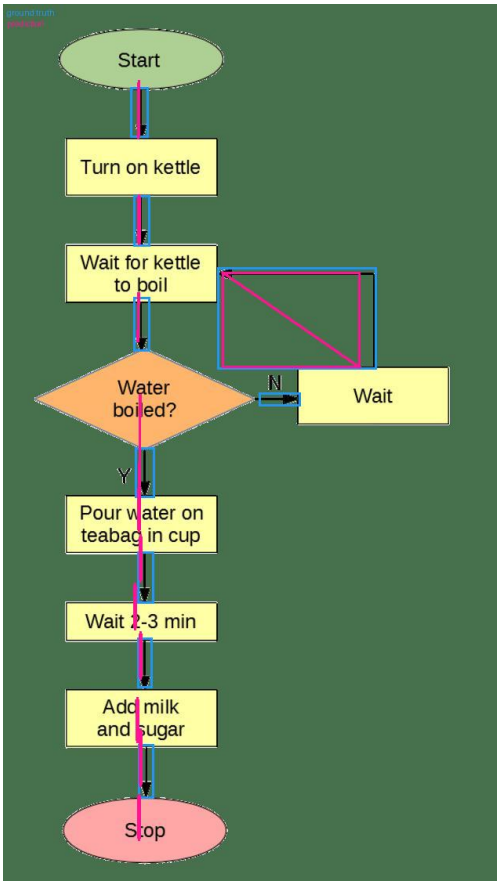


Figure 3: A good prediction (pink) on a simple linear flowchart, almost perfectly on top of the ground truth (blue). All the straight arrows are correct, the only mistake is on the feedback loop at the top right (N → Wait → Water boiled?), where the model draws a diagonal instead of following the real path. This is the same kind of error shown in Figure 4.

5 Evaluation on dataset UniFi

The UniFi dataset is a more varied collection of diagrams including hand-drawn ones, multiple charts in the same image, and other complex diagrams to decipher. The dataset doesn't have a precise labeling yet, especially not the tail/head format used in this project. For this reason I used a YOLO model to automatically label the points of the arrows. The goal was to test the model on a more challenging dataset and potentially use it in finetuning to improve its performance.

It's important to note that, even though YOLO is a good detector, it's not perfect and the labels produced clearly contained errors. I still tried using them to see if the model could learn from them, but the results were not as good as I hoped.

Figures 6 and 7 show two examples of the automatic labels: one where all the arrows are labelled correctly and one with some of the typical errors.

I split the 353 labelled images into training and test sets with an 80/20 split, obtaining 282 training images and 71 test images, and trained three different adapters: the final FC-Detection adapter as it is, the same adapter fine-tuned for 3 more epochs on the UniFi training set, and a new adapter trained from scratch only on UniFi for 10 epochs. The results are shown in Table 3.

The results are worse than I expected (Table 3). The general trend is that fine-tuning on UniFi makes the model more cautious: it produces fewer false positives, but it also finds fewer arrows, so the F1 stays practically the same. Compared to FC-Detection, though, the precision drops drastically, from about 0.47 to between 0.17 and 0.27. My guess is that this comes from the combination of the noise in the automatic labels and the additional complexity of these images, with their icons, crowded layouts and small arrows that are hard to read at 384 pixels.

6 Training with boxes on UniFi

Since the previous attempt at using the UniFi dataset didn't work as expected, further changes were made toimprove the model's performance. The first change was to clean the labels, as I noticed that the automatic labels contained many errors. The second change was to introduce a box metric, which is less strict than the point metric.

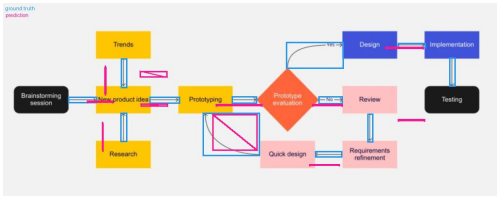


Figure 4: A harder case. Most of the horizontal arrows between the main boxes are correct, but the vertical arrows at the top left are not aligned with the ground truth, the curved feedback arrow at the top (Design → Prototype evaluation) is not found at all, and a couple of predicted arrows don't correspond to any real arrow.

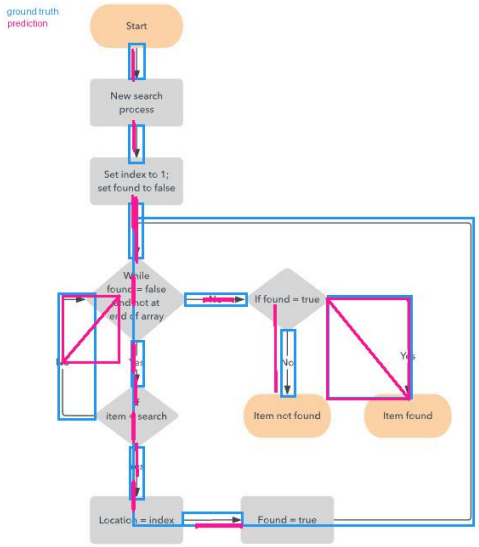


Figure 5: The same flowchart as in Figure 2, this time with the full VLLM pipeline. Almost all the straight arrows are correct, but the arrows of the nested loops are often drawn as a diagonal instead of following the real path. Unlike YOLO the model doesn't miss these arrows, it just doesn't place them precisely.

6.1 Label cleaning and box metric

The automatic labels produced in the previous section needed some cleaning, as can be seen by the results obtained. The main problem was that YOLO sometimes detected arrows that didn't exist, or count some arrows more than once. I automatically cleaned the dataset using the rules described in this section: I extracted the drawing from the background, taking the most common colour as background. An arrow is removed if less than 75% of the path between its tail and head, straight or with an elbow, lies on drawn pixels and the two ends are not connected inside its box. Two arrows whose tails are closer than 3% of the image diagonal and that point in opposite directions are merged into a single double-headed arrow, since YOLO often splits these in two. Finally the arrows are visited by decreasing confidence, and one is removed as a duplicate if it is almost parallel to an arrow already kept, shares one of its ends and has at least 75% of its segment inside that arrow's box (Figures 8 and 9).

I also added a metric on whole arrows: every ground truth arrow gets a box between tail and head, enlarged by 3% of the image size, and a prediction is a true positive if at least 75% of its segment falls inside a free box. This is less sensitive to label errors than the point metric, and the direction is counted separately.

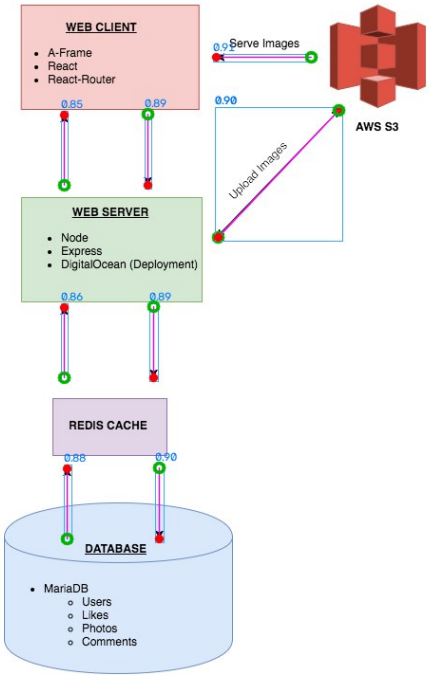


Figure 6: Automatic YOLO labels on a UniFi diagram (green ring = tail, red dot = head). All the arrows are labelled correctly, including the diagonal one and the horizontal arrow at the top.

Adapter	Inference	Prec.	Recall	F1
FC-Detection	greedy	0.17	0.26	0.208
+ UniFi fine-tuning	greedy	0.27	0.18	**0.214**
UniFi only	self-cons.	0.22	0.10	0.139

Table 3: Results on the UniFi test set (71 images, automatic labels), with the same metric as Table 1.

6.2 Training with boxes

Starting from the best FC-Detection adapter I had, the model samples 3 answers per training image at temperature 0.7. The target of every ground truth arrow is the average of the samples that fall inside its box and lie within 3% of the image diagonal from the label, or the label itself if no sample finds it. The model is then fine-tuned on these targets with cross entropy plus NTL for 3 epochs, so it learns coordinates it can already produce.

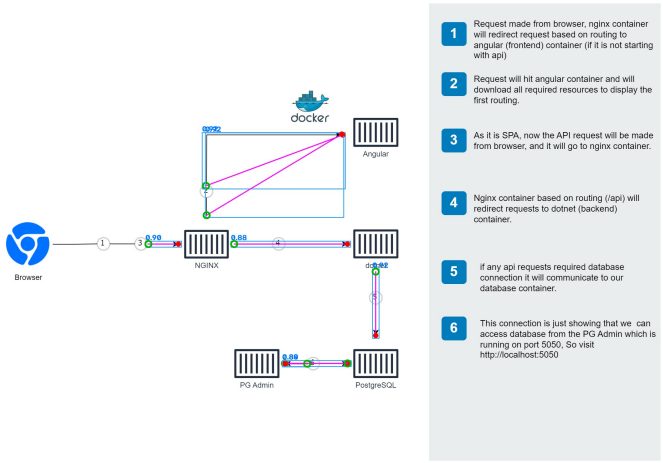


Figure 7: Automatic YOLO labels with errors. The elbow connector at the top is detected twice and approximated with two diagonals, and the double-headed arrow between PG Admin and PostgreSQL is split, with a tail placed in the middle of the line.

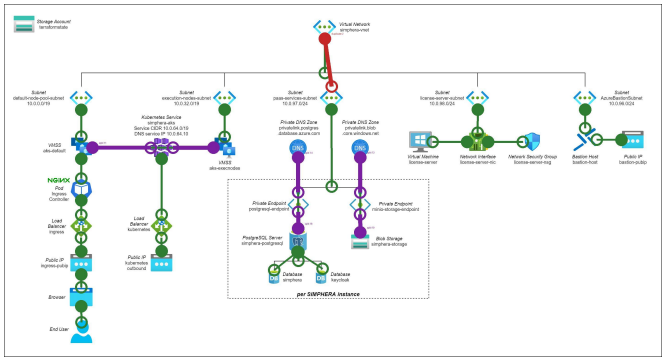


Figure 8: Label cleaning (green = kept, purple = merged, red = removed). Six split double-headed arrows are merged back and one duplicate is removed.

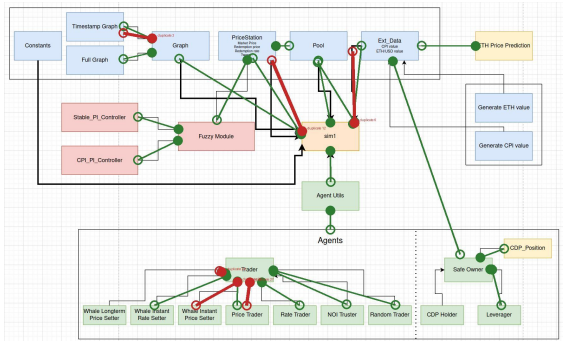


Figure 9: Six elbow connectors detected twice by YOLO, with the copies (red) removed.

6.3 Parsing, resolution and order of the arrows

Greedy decoding often fell into loops repeating the same arrow until the token limit, leaving the JSON unclosed on many test images. I made the parsing recover the complete objects and discard repeated arrows, and stopped the generation when the last 12 arrows are all repetitions.

Since I was working with a bigger GPU I moved to 768 pixels as input resolution for the images, this doubled the box F1 of the FC-Detection adapter on 20 test images without retraining, so I used it from then on. Finally I sorted the target arrows by the position of their head, top to bottom and left to right, instead of the arbitrary order of the label file. "Box raster" is the adapter trained at 768 pixels with this order, "Box" the first one at 384 pixels.

6.4 Results

Adapter	Res.	Box P	Box R	Box F1	PCK F1
FC-Detection	384	0.205	0.331	0.253	0.209
Box	384	0.228	0.190	0.207	0.184
FC-Detection	768	0.362	**0.522**	0.428	0.379
Box	768	0.454	0.294	0.357	0.319
Box raster	768	**0.549**	0.354	**0.430**	**0.384**

Table 4: Results on the cleaned UniFi test set (71 images, 701 arrows). Rows at 768 pixels use the robust parsing.

Adapter	Box P	Box R	Box F1	PCK F1
FC-Detection	0.644	**0.765**	0.699	0.654
Box	0.752	0.638	0.691	0.669
Box raster	**0.838**	0.644	**0.728**	**0.701**

Table 5: Results on the FC-Detection validation set (30 images, 362 arrows) at 768 pixels.

The resolution had a huge effect on the accuracy of the model (Table 4): the FC-Detection adapter goes from a box F1 of 0.253 to 0.428 on UniFi without retraining. The raster adapter reaches the same F1 with about a third of the false positives (204 against 644), but finds fewer arrows.

The most interesting result is testing this new adapter on FC-Detection (Table 5): the raster adapter, trained further only on UniFi, is the best on every F1 and lowers the false positives from 153 to 45. Its PCK F1 of 0.701 is the best of the whole project, against 0.534 for the pipeline with self-consistency (Table 1). Part of the gain comes from resolution and parsing, since the FC-Detection adapter alone reaches 0.654, but my guess is that the crowded diagrams of UniFi also teach the model to be more careful. Figure 10 shows two predictions.

7 Scaling to a bigger model

To see if the limits of the pipeline come from the size of the model, I repeated the main experiments with Qwen3-VL-32B-Instruct, four times bigger than the 8B model

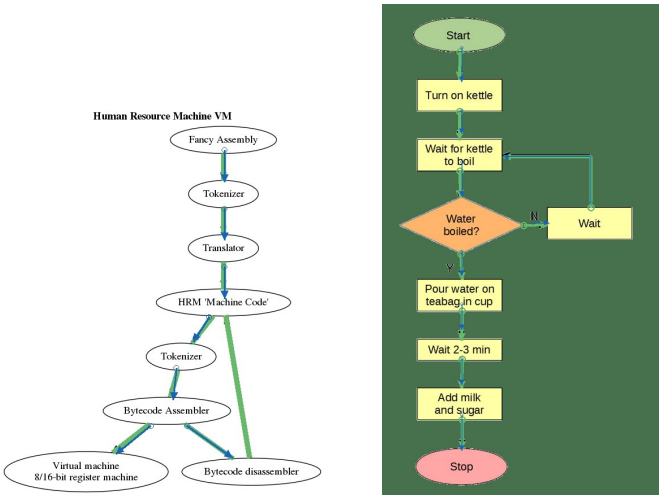


Figure 10: Raster adapter (blue) over the labels (green). Left: UniFi, seven of eight arrows found. Right: the flowchart of Figure 3, all nine arrows found.

used so far. I ran it zero-shot and then fine-tuned it with exactly the same recipe as the final 8B adapter: same data, two stages with 5 epochs on FlowLearn and 3 on FC-Detection, LoRA rank 48 and NTL. The 8B reference in Table 6 was evaluated with the same settings as the fine-tuned 32B.

Configuration	Prec.	Recall	F1
32B zero-shot	0.333	0.150	0.207
32B + LoRA r=48	**0.633**	0.411	**0.507**
8B + LoRA r=48	0.468	**0.414**	0.439

Table 6: Qwen3-VL-32B against the 8B model on the FC-Detection validation set, greedy decoding.

Zero-shot, the 32B model is clearly better than the 8B one, wich is expected and would confirm the hypothesis that bigger models are better at spatial tasks like this one. Fine tuning also has a similar effect on both models, with the 32B model improving by a similar margin that the 8B one but reaching a better performance, which is also expected since it has more capacity to learn.

8 Conclusions

By putting together LoRA fine-tuning, the Number Token Loss and self-consistency I was able to take the model from an F1 of 0.097 in zero-shot to 0.534, more than five times better. LoRA fine-tuning and the NTL are both essential for the results obtained (without the NTL the F1 is halved), and self-consistency added another +16% at inference time without any retraining. The main result of this project is that a small open model without any adaptation is not usable for a task like this, but with a few techniques that are relatively cheap and well known it becomes a model that works reasonably well, without making the model any bigger. This shows that big improvements are possible on VLLMs even for a task like this one, that these models are not built for. On the more varied UniFi diagrams the first attempt with noisy automatic labels stayed around an F1 of 0.21. After cleaning the labels, training on the model's own samples and moving to 768 pixels, the raster adapter reaches a box F1 of 0.430 on UniFi and a PCK F1 of 0.701 on FC-Detection, the best result of the project.

The model still has problems with curved and diagonal arrows, especially with the feedback loops that go back to an earlier step of the flowchart. Straight arrows, both vertical and horizontal, are now detected well (Figure 3), but more complex arrows are still a problem, especially in flowcharts with many nested loops (Figures 4 and 5). My guess is that it's hard to describe a curved arrow using only the coordinates of its tail and head, so the model tends to approximate it with a straight line.

The experiment with Qwen3-VL-32B shows that the same recipe scales to a bigger model, which reaches an F1 of 0.507 with greedy decoding against the 0.439 of the 8B one. A natural next step would be to test more model sizes, to understand which is the minimum number of parameters that allows to get a detection comparable to a specialized detector like YOLO, while keeping the flexibility of a VLLM. With more time and compute I would also try to add a reinforcement learning stage after the fine-tuning, using a technique like GRPO or PPO. This way the model could be trained directly on the point-distance metric used for the evaluation, instead of using the loss on the tokens as an approximation of it.

References

[1] E. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, W. Chen. *LoRA: Low-Rank Adaptation of Large Language Models*. ICLR, 2022.

[2] J. Zausinger, L. Pennig, A. Kozina, S. Sdahl, J. Sikora, A. Dendorfer, T. Kuznetsov, M. Hagog, N. Wiedemann, K. Chlodny, V. Limbach, A. Ketteler, T. Prein, V. M. Singh, M. Danziger, J. Born. *Regress, Don't Guess – A Regression-like Loss on Number Tokens for Language Models*. ICML, 2025.

[3] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, D. Zhou. *Self-Consistency Improves Chain of Thought Reasoning in Language Models*. ICLR, 2023.

[4] J. Pan et al. *FlowLearn: Evaluating Large Vision-Language Models on Flowchart Understanding*. ECAI, 2024.

[5] galirage/FC-Detection dataset. HuggingFace Datasets. `https://huggingface.co/datasets/galirage/FC-Detection`

[6] Creating and Validating a Ground Truth Dataset of Unified Modeling Language Diagrams Using Deep Learning Techniques. *Applied Sciences*, 14(23):10873, 2024. https://www.mdpi.com/2076-3417/14/23/10873

[7] shreyanshu09/block_diagram_symbol_detection model. HuggingFace Models. https://huggingface.co/shreyanshu09/block_diagram_symbol_detection

[8] S. Bhushan, E.-S. Jung, M. Lee. *Unveiling the Power of Integration: Block Diagram Summarization through Local-Global Fusion*. Findings of ACL, 2024.